
ST5039CMD · OPERATING SYSTEM

Lab 3

Investigating Process Lifecycles and OS Interaction

Lab notes, commands and evidence

Student Name **Gaurav Poudel**
Module **Programming and Operating System**

Overview

This lab contains the program files, lab instructions, and notes on process lifecycle and OS interaction with the help of C program.

LEARNING OBJECTIVES

1. Explain how OS keeps track of PIDs and PPIDs
2. Analyze about exit codes and its importance
3. Map the gap between high level C program and low level OS process management
4. Demonstrate about input and output streams (stdin and stdout)

CONTENTS

Task 1	The Long Running Process
Task 2	Process Identity (PID & PPID)
Task 3	Exit Codes & OS Feedbacks
Task 4	Standard I/O Streams
Task 5	Conditional Execution and Termination

Task 1: The Long Running Process

CONCEPT

In this lab, we'll understand the concept of PID and practically demonstrate how OS keeps track of process ID and look how it can be seen while process is actually running in the background.

Definition: PID stands for Process ID and it is a unique identifier for each process that is currently at running state.

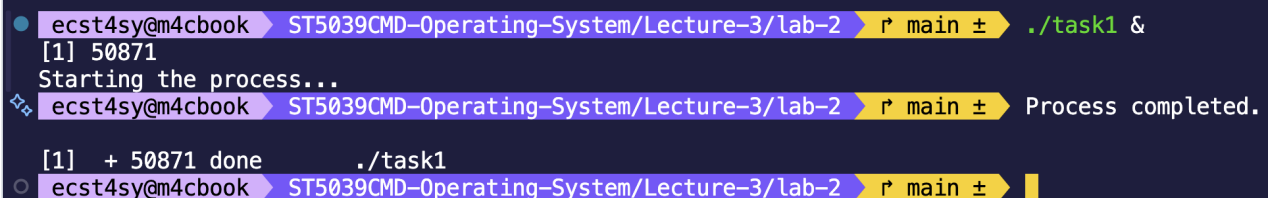
COMMANDS

```
# Compile the source code
gcc task1_alive.c -o task1

# Run the executable in the background
./task1 &

# Monitor the process for PID
ps aux | grep 'task1'
```

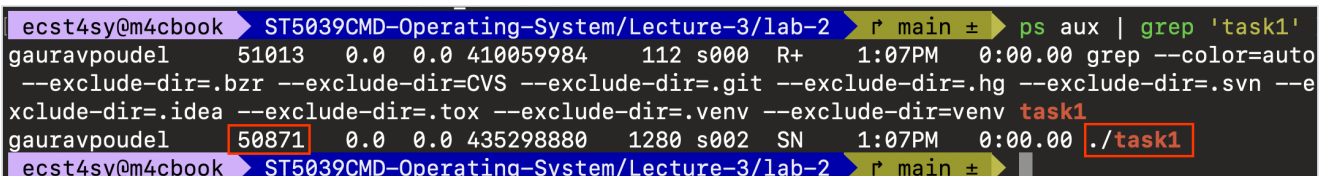
EVIDENCES



```
ecst4sy@m4cbook ST5039CMD-Operating-System/Lecture-3/lab-2 r main ± ./task1 &
[1] 50871
Starting the process...
ecst4sy@m4cbook ST5039CMD-Operating-System/Lecture-3/lab-2 r main ± Process completed.
[1] + 50871 done ./task1
ecst4sy@m4cbook ST5039CMD-Operating-System/Lecture-3/lab-2 r main ±
```

Figure 1.1 — The process running in the background

Above it can be noticed that process is running and exited successfully, meanwhile we have verified it with `ps aux | grep 'task1'`



```
ecst4sy@m4cbook ST5039CMD-Operating-System/Lecture-3/lab-2 r main ± ps aux | grep 'task1'
gauravpoudel 51013 0.0 0.0 410059984 112 s000 R+ 1:07PM 0:00.00 grep --color=auto
--exclude-dir=.bzip --exclude-dir=CVS --exclude-dir=.git --exclude-dir=.hg --exclude-dir=.svn --e
xclude-dir=.idea --exclude-dir=.tox --exclude-dir=.venv --exclude-dir=.venv task1
gauravpoudel 50871 0.0 0.0 435298880 1280 s002 SN 1:07PM 0:00.00 ./task1
ecst4sy@m4cbook ST5039CMD-Operating-System/Lecture-3/lab-2 r main ±
```

Figure 1.2 — Verifying the process with `ps aux`

- **PID:** 50871 (assigned to the `task1` process)

Task 2: Process Identity (PID & PPID)

CONCEPT

When we talk about linux system, each process is assigned with PID as we have already seen in the above task, but there also exist parent process ID (PPID) which is the process that created the child process (in most cases PPID is our *shell*).

PPID: It is a ID for process which created a child process.

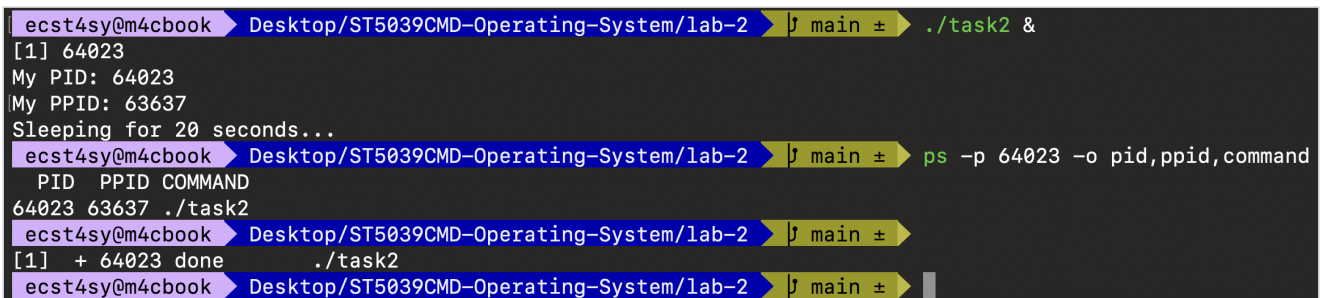
COMMANDS

```
# Compile the source code
gcc task2_identity.c -o task2

# Run the program in the background
./task2 &

# Verify it
ps -p [PID] -o pid,ppid,command
```

EVIDENCES



```
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 j main ± ./task2 &
[1] 64023
My PID: 64023
My PPID: 63637
Sleeping for 20 seconds...
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 j main ± ps -p 64023 -o pid,ppid,command
  PID  PPID  COMMAND
64023 63637 ./task2
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 j main ±
[1] + 64023 done ./task2
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 j main ±
```

Figure 2.1 — PID and PPID reported by the program and by `ps`

Above it is verified that both PID and PPID are identical when `getpid()` retrieves it and when we check it against the output of `ps` command.

Task 3: Exit Codes & OS Feedbacks

CONCEPT

When program completes what it was supposed to do, it returns a integer value to **OS** via `main()` making sure if the program succeeded or failed. `0` refers to that program successfully completed and any non-zero value refers to failure. This is important because in many cases we use return values to determine if it should be continued or abort (we also do branching based on return values sometimes *more on that later*).

Definition: Exit Codes basically is a integer value that determine success or failure.

COMMANDS

```
# Compile the source code
gcc task3_exit.c -o task3

# Run the program
./task3

# Verify it
echo $?
```

EVIDENCES

```
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 } main ± gcc task3_exit.c -o task3
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 } main ± ./task3
Enter a number (positive for success and negative for failure): 16
Exited Successfully
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 } main ± echo $?
0
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 } main ± ./task3
Enter a number (positive for success and negative for failure): -3
Exited with Failure
x ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 } main ± echo $?
1
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 } main ±
```

Figure 3.1 — Exit code returned to the OS

In the figure above, exit codes are verified with `echo $?` commands because everytime the program finishes running, it stores the return value to `$?`

Task 4: Standard I/O Streams

CONCEPT

This task shows that OS provides two different streams i.e. `stdin` and `stdout`. It is a communication channel that OS provides to computer program so that it can send and receive data.

`stdin`: This is a input stream where a program reads data.

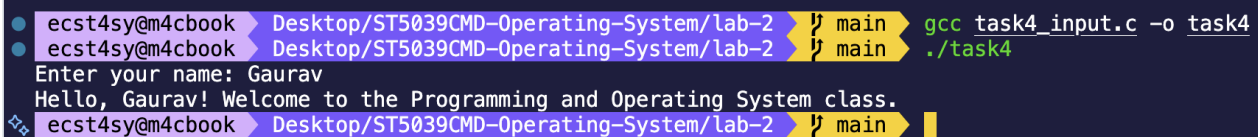
`stdout`: This is a output stream where a program writes data.

COMMANDS

```
# Compile the source code
gcc task4_input.c -o task4

# Run the program
./task4
```

EVIDENCES



```
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 ? main gcc task4_input.c -o task4
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 ? main ./task4
Enter your name: Gaurav
Hello, Gaurav! Welcome to the Programming and Operating System class.
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 ? main
```

Figure 4.1 — Reading from `stdin` and writing to `stdout`

As I have already explain enough about input and output streams above. So lets talk about the image above, where it asks for a name i.e. **`stdin`** and outputs it as a string i.e. **`stdout`**

Task 5: Conditional Execution and Termination

CONCEPT

This task shows that program takes different paths based on **user input** and what is its exit codes. This is important because if there's automated script or pipeline where exit codes from previous operation determine whether the pipeline/script should continue or halt.

COMMANDS

```
# Compile the source code
gcc task5_control.c -o task5

# Run the program
./task5

# Verify it
echo $?
```

EVIDENCES

```
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 main ± gcc task5_control.c -o task5
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 main ± ./task5
Process ID: 77898
Do you want to continue? (1 for continuing and 0 for exiting): 1
Continuing...
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 main ± echo $?
0
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 main ± ./task5
Process ID: 78124
Do you want to continue? (1 for continuing and 0 for exiting): 0
Exiting...
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 main ± echo $?
1
ecst4sy@m4cbook Desktop/ST5039CMD-Operating-System/lab-2 main ±
```

Figure 5.1 — Different paths and exit codes based on user input

As it can be noticed that when input `1` is passed, it does continue and return the exit code of `0` (success), in contrast, when the input is `anything other than 1` it simply exits and returns the exit code `1` (failure). This is how automated pipelines/script can be controlled based on user input and exit codes.

Author

Student Name: Gaurav Poudel

Module: Programming and Operating System